# Hướng dẫn code: Mail & Phân quyền — MetroTicket API

> Dự án: `metroticket-api` · Java 25 · Spring Boot 4.1.1 · PostgreSQL · Flyway
> Ngày lập: 21/10/2026

Tài liệu gồm hai phần:

1. **Đánh giá hiện trạng**: các file đã tạo trong repo có chạy ổn định không.
2. **Hướng dẫn code chi tiết** cho hai chức năng:
   - **Mail**: gửi email qua SMTP, nội dung dựng từ template, gửi bất đồng bộ; dùng để gửi mã xác thực (OTP) khi đăng ký và quên mật khẩu.
   - **Phân quyền (RBAC)**: người dùng, vai trò, quyền; đăng nhập bằng JWT; chặn truy cập theo URL và theo method.

Toàn bộ code trong tài liệu đã được **biên dịch và chạy test thực tế** (7/7 test pass) trên một bản sao của dự án, với PostgreSQL thật (Testcontainers) và SMTP giả (GreenMail).

---

## Mục lục


1. Đánh giá hiện trạng
2. Kiến trúc tổng thể
3. Chuẩn bị môi trường và cấu hình
4. Module Mail
5. Module Verification (mã xác thực qua email)
6. Phân quyền (RBAC + JWT)
7. API đăng ký, đăng nhập, quản trị
8. Database migration (Flyway)
9. Kiểm thử
10. Thứ tự thực hiện và checklist
11. Hạn chế và hướng mở rộng


---

## 1. Đánh giá hiện trạng

### 1.1 Kết luận

**Hiện tại chưa ổn định: dự án chưa build/khởi động được và các chức năng mail, phân quyền chưa có logic.**

- Tất cả 11 class Java trong `mail/`, `verification/`, `config/` chỉ là **khung rỗng** (`public class X { }`).
- **Chưa có code phân quyền**: không có `SecurityConfig`, không có entity User/Role. Thư mục `db/migration` và `templates` còn trống.
- Chạy `mvn test` thì **thất bại**:

```
APPLICATION FAILED TO START
Reason: Failed to determine a suitable driver class
Tests run: 1, Failures: 0, Errors: 1  →  BUILD FAILURE
```

Nguyên nhân: `pom.xml` có `data-jpa` + `flyway` + `postgresql` nhưng `application.properties` không khai báo `spring.datasource.*`.

## 1.2 Đánh giá từng file

| File | Trạng thái | Ghi chú |
|---|---|---|
| `mail/ MailMessage.java` | [!] Rỗng | Cần là `record` chứa to/subject/text/html |
| `mail/ MailProps.java` | [!] Rỗng | Cần `@ConfigurationProperties("app.mail")` |
| `mail/ MailTemplates.java` | [!] Rỗng | Cần dựng nội dung bằng Thymeleaf |
| `mail/ Mailsender.java` | [!] Rỗng + sai quy ước tên | Đổi thành **`MailSender`** (interface) |
| `mail/ SmtpMailSender.java` | [!] Rỗng | Cài đặt `MailSender` bằng `JavaMailSender` |
| `verification/ EmailVerficationChallenge.java` | [!] Rỗng + **sai chính tả** | Đổi thành **`EmailVerificationChallenge`** (thiếu chữ "i") |
| `verification/ EmailVerificationChallengeRepository.java` | [!] Rỗng | Cần là interface `JpaRepository` |
| `verification/ VerificationCodeIssued.java` | [!] Rỗng | Cần là `record` sự kiện |
| `verification/ VerificationCodeService.java` | [!] Rỗng | Logic cấp/kiểm tra mã |
| `verification/ VerificationMailListener.java` | [!] Rỗng | Nghe sự kiện, gửi mail |
| `verification/ VerificationsProps.java` | [!] Rỗng + sai tên | Đổi thành **`VerificationProps`** |
| `config/ AsyncConfig.java` | [!] Rỗng | Cần `@EnableAsync` + thread pool |
| `application.properties` | [!] Thiếu | Thiếu datasource, JWT; `app.*` chưa class nào đọc |
| `test/ MetroticketApiApplicationTests.java` | [X] Fail | Context không khởi động được |
| `.mvn/ wrapper/` | [X] **Thiếu** | `./ mvnw` báo lỗi `cannot read distributionUrl` → **CI (`./ mvnw -B verify`) sẽ fail** |
| `docker-compose.yml` | [!] | Chưa có service **PostgreSQL**; `minio:latest`, `mailpit:latest` không cố định phiên bản |
| `.github/ workflows/ backend-ci.yml` | [X] | `trivy-action@ <commit-sha-da-kiem-tra>` là placeholder → workflow lỗi. Bước "Testcontainers + GreenMail + RBAC" nhưng pom chưa có các dependency này |
| `.github/ workflows/ publish.yml` | [X] | `ghcr.io/ <owner-viet-thuong>/ ...` là placeholder |
| `.gitignore` | [!] | Thiếu `target/` , `.DS_Store`, `.idea/` (hiện chỉ có trong `.dockerignore`) |
| `Dockerfile` | [OK] | Ổn: multi-stage, layered jar, user không phải root, healthcheck |

| File | Trạng thái | Ghi chú |
|---|---|---|
| `.env.example` | [!] | Thiếu `DB_*`, `JWT_SECRET`, `ADMIN_*` |

### 1.3 Việc cần sửa ngay (ngoài phần code mới)

```
# 1. Tạo lại Maven wrapper (cần Maven cài sẵn)
mvn -N wrapper:wrapper

# 2. Đổi tên file sai quy ước
git mv src/main/java/com/metroticket_api/mail/Mailsender.java \
       src/main/java/com/metroticket_api/mail/MailSender.java
git mv src/main/java/com/metroticket_api/verification/EmailVerficationChallenge.java \
       src/main/java/com/metroticket_api/verification/EmailVerificationChallenge.java
git mv src/main/java/com/metroticket_api/verification/VerificationsProps.java \
       src/main/java/com/metroticket_api/verification/VerificationProps.java

# 3. Bổ sung .gitignore
printf "target/\n.DS_Store\n.idea/\n" >> .gitignore
```

> Repo hiện **chưa phải git repository** (không có `.git`). Nếu chưa init thì dùng `mv` thay cho `git mv`.

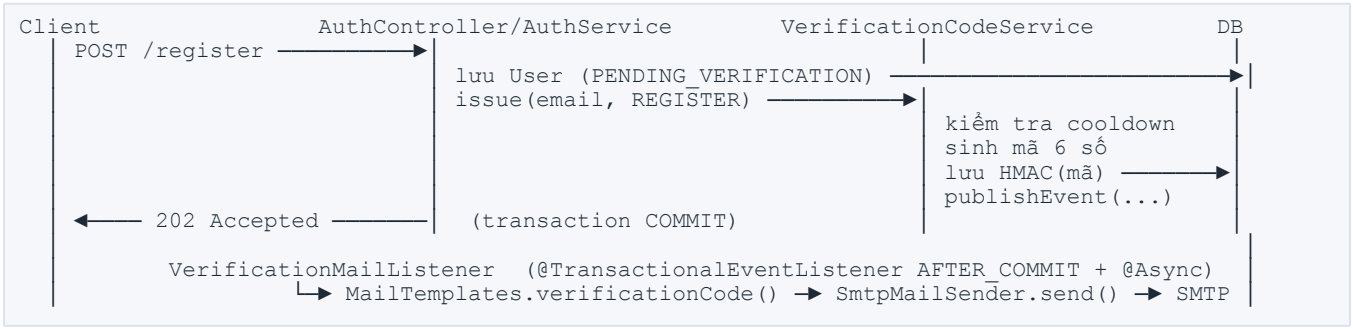
---

## 2. Kiến trúc tổng thể

### 2.1 Cấu trúc package sau khi hoàn thành

```
com.metroticket_api
├── MetroticketApiApplication.java      (+ @ConfigurationPropertiesScan)
├── config/
│   ├── AsyncConfig.java                thread pool "mailExecutor"
│   ├── TimeConfig.java                 bean Clock + @EnableScheduling
│   └── ApiExceptionHandler.java        trả lỗi dạng ProblemDetail
├── mail/                               (gửi mail, không biết gì về nghiệp vụ)
│   ├── MailMessage.java
│   ├── MailProps.java
│   ├── MailSender.java                 interface
│   ├── SmtpMailSender.java             cài đặt bằng SMTP
│   ├── MailSendException.java
│   └── MailTemplates.java              dựng nội dung từ Thymeleaf
├── verification/                       (mã OTP qua email)
│   ├── VerificationProps.java
│   ├── VerificationPurpose.java        REGISTER | RESET_PASSWORD
│   ├── VerificationResult.java
│   ├── EmailVerificationChallenge.java entity
│   ├── EmailVerificationChallengeRepository.java
│   ├── VerificationCodeIssued.java     sự kiện
│   ├── VerificationCodeService.java
│   ├── VerificationMailListener.java
│   ├── VerificationCleanupJob.java
│   └── ResendTooSoonException.java
├── user/                               (người dùng & vai trò)
│   ├── Permission.java  Role.java  UserStatus.java
│   ├── User.java  UserRepository.java  UserDtos.java
│   ├── MeController.java               GET /api/me
│   └── AdminUserController.java        /api/admin/users
├── security/                           (JWT + cấu hình bảo mật)
│   ├── JwtProps.java  JwtConfig.java  TokenService.java
│   ├── RoleAuthoritiesConverter.java
│   ├── SecurityConfig.java
│   ├── AdminBootstrap.java
│   └── CurrentUser.java
└── auth/                               (đăng ký, đăng nhập)
    ├── AuthDtos.java  AuthException.java
    ├── AuthService.java
    └── AuthController.java
```

### 2.2 Luồng đăng ký và gửi mã xác thực




Ba nguyên tắc thiết kế:

1. **Chỉ gửi mail sau khi DB commit.** Nếu lưu thất bại thì không gửi mã "ma" cho người dùng.
2. **Gửi bất đồng bộ.** API trả về ngay, không chờ SMTP (Gmail có thể mất vài giây).
3. **Không lưu mã gốc.** DB chỉ chứa HMAC-SHA256 của mã. Lộ DB cũng không dùng được mã.

### 2.3 Luồng phân quyền

```
POST /api/auth/login → kiểm tra mật khẩu (BCrypt) → TokenService phát JWT HS256
                                                    { sub: <userId>, roles: ["STAFF"], exp:
                                                            +15m }

GET /api/staff/... + "Authorization: Bearer <jwt>"
   └→ BearerTokenAuthenticationFilter → JwtDecoder (kiểm tra chữ ký, exp, issuer)
         └→ RoleAuthoritiesConverter: roles=["STAFF"] → ROLE_STAFF, TICKET_VALIDATE, ...
               └→ Lớp 1: SecurityFilterChain   (/api/staff/** cần ROLE_STAFF hoặc ROLE_ADMIN)
                     └→ Lớp 2: @PreAuthorize    (hasAuthority('TICKET_VALIDATE'), quyền sở hữu...)
```

---

## 3. Chuẩn bị môi trường và cấu hình

### 3.1 `pom.xml`: thêm dependency test

Các starter chính (`mail`, `security`, `oauth2-resource-server`, `thymeleaf`, `data-jpa`, `flyway`, `validation`) **đã có sẵn**. Chỉ cần thêm phần test vào trong `<dependencies>`:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-testcontainers</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-postgresql</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-junit-jupiter</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>com.icegreen</groupId>
    <artifactId>greenmail-junit5</artifactId>
    <version>2.1.5</version>
    <scope>test</scope>
</dependency>
```

> Spring Boot 4 dùng Testcontainers 2.x: artifact đổi tên thành `testcontainers-postgresql`, class nằm ở `org.testcontainers.postgresql.PostgreSQLContainer`.

## 3.2 `.env.example`: bổ sung biến

```
# Database
DB_URL=jdbc:postgresql://localhost:5432/metroticket
DB_USERNAME=metroticket
DB_PASSWORD=doi-mat-khau-nay

# Mail (Gmail: dùng App Password 16 ký tự, KHÔNG dùng mật khẩu đăng nhập)
MAIL_USERNAME=
MAIL_PASSWORD=
SMTP_FROM_ADDRESS=email-cua-ban@gmail.com
SMTP_FROM_NAME=MetroTicket
EMAIL_VERIFICATION_HASH_KEY=chuoi-ngau-nhien-tu-openssl

# JWT & admin đầu tiên
JWT_SECRET=chuoi-ngau-nhien-tu-openssl
ADMIN_EMAIL=admin@metroticket.vn
ADMIN_PASSWORD=doi-ngay-sau-lan-dang-nhap-dau

# Spring AI (nếu dùng)
ANTHROPIC_API_KEY=
```

Sinh khoá ngẫu nhiên (tối thiểu 32 ký tự):

```
openssl rand -base64 48
```

## 3.3 `docker-compose.yml`: thêm PostgreSQL

```
  postgres:
    image: postgres:17-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: metroticket
      POSTGRES_USER: ${DB_USERNAME}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USERNAME} -d metroticket"]
      interval: 5s
      retries: 10
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

Nhớ thêm `postgres_data:` vào mục `volumes:`. Trong service `backend`, thêm `DB_URL: jdbc:postgresql://postgres:5432/metroticket` và `depends_on: postgres: condition: service_healthy`.

**Dev không cần Gmail:** chạy `docker compose --profile mailpit up -d`, rồi đặt `MAIL_HOST=localhost`, `MAIL_PORT=1025`, `MAIL_SMTP_AUTH=false`, `MAIL_STARTTLS=false`. Mở http://localhost:8025 để xem mail.

## 3.4 `application.properties` (thay toàn bộ)

**`src/main/resources/application.properties`**

```
spring.application.name=metroticket-api

# ===== Database (PostgreSQL + Flyway) =====
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/metroticket}
spring.datasource.username=${DB_USERNAME:metroticket}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false

# ===== Mail (SMTP) =====
spring.mail.host=${MAIL_HOST:smtp.gmail.com}
spring.mail.port=${MAIL_PORT:587}
spring.mail.default-encoding=UTF-8
spring.mail.username=${MAIL_USERNAME}
spring.mail.password=${MAIL_PASSWORD}
```

```
spring.mail.properties.mail.smtp.auth=${MAIL_SMTP_AUTH:true}
spring.mail.properties.mail.smtp.starttls.enable=${MAIL_STARTTLS:true}
spring.mail.properties.mail.smtp.starttls.required=${MAIL_STARTTLS:true}
spring.mail.properties.mail.smtp.connectiontimeout=5000
spring.mail.properties.mail.smtp.timeout=5000
spring.mail.properties.mail.smtp.writetimeout=5000

app.mail.from=${SMTP_FROM_ADDRESS}
app.mail.from-name=${SMTP_FROM_NAME:MetroTicket}

# ===== Mã xác thực email =====
app.verification.hash-key=${EMAIL_VERIFICATION_HASH_KEY}
app.verification.lifetime=10m
app.verification.resend-cooldown=60s
app.verification.max-failed-attempts=5

# ===== JWT & phân quyền =====
app.jwt.secret=${JWT_SECRET}
app.jwt.issuer=metroticket-api
app.jwt.access-token-ttl=15m
app.bootstrap.admin-email=${ADMIN_EMAIL:}
app.bootstrap.admin-password=${ADMIN_PASSWORD:}

# ===== Spring AI (đang có trong pom) =====
spring.ai.anthropic.api-key=${ANTHROPIC_API_KEY:chua-cau-hinh}

# ===== Actuator =====
management.endpoints.web.exposure.include=health
management.health.mail.enabled=true
```

Giải thích các điểm chính:

- `ddl-auto=validate`: Hibernate **không tự tạo bảng**, chỉ kiểm tra entity khớp với schema do Flyway tạo.
- `open-in-view=false`: tránh giữ kết nối DB suốt request (mặc định Spring bật và có cảnh báo).
- Biến dạng `${MAIL_HOST:smtp.gmail.com}`: lấy từ môi trường, nếu không có thì dùng giá trị sau dấu `:`.
- `spring.ai.anthropic.api-key` có giá trị mặc định để ứng dụng không lỗi khi chưa dùng AI.

### 3.5 Class khởi động

**src/main/java/com/metroticket_api/MetroticketApiApplication.java**

```java
package com.metroticket_api;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;

@SpringBootApplication
@ConfigurationPropertiesScan // tự đăng ký MailProps, VerificationProps, JwtProps
public class MetroticketApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(MetroticketApiApplication.class, args);
    }

}
```

`@ConfigurationPropertiesScan` tự tìm và đăng ký mọi `record` có `@ConfigurationProperties`.

---

## 4. Module Mail

**Mục tiêu:** cung cấp một interface `MailSender` đơn giản cho toàn hệ thống. Nghiệp vụ (verification, sau này là gửi vé, hoá đơn...) không cần biết SMTP hoạt động thế nào.

### 4.1 `config/AsyncConfig.java`: thread pool gửi mail

**src/main/java/com/metroticket_api/config/AsyncConfig.java**

```java
package com.metroticket_api.config;

import java.util.concurrent.Executor;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

/**
 * Bật @Async và tạo một thread pool riêng cho việc gửi mail,
 * để request HTTP không phải chờ SMTP (có thể mất 1-5 giây).
 */
@Configuration
@EnableAsync
public class AsyncConfig {

    public static final String MAIL_EXECUTOR = "mailExecutor";

    @Bean(name = MAIL_EXECUTOR)
    public Executor mailExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(4);
        executor.setQueueCapacity(500);
        executor.setThreadNamePrefix("mail-");
        // Khi tắt ứng dụng, chờ các mail đang gửi dở hoàn thành
        executor.setWaitForTasksToCompleteOnShutdown(true);
        executor.setAwaitTerminationSeconds(20);
        executor.initialize();
        return executor;
    }
}
```

- Dùng pool riêng để mail chậm không chiếm thread của việc khác.
- `queueCapacity=500`: khi SMTP chậm, tối đa 500 mail chờ trong hàng đợi.
- `waitForTasksToCompleteOnShutdown`: không mất mail đang gửi dở khi redeploy.

### 4.2 `config/TimeConfig.java`: bean `Clock`

**`src/main/java/com/metroticket_api/config/TimeConfig.java`**

```java
package com.metroticket_api.config;

import java.time.Clock;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.EnableScheduling;

@Configuration
@EnableScheduling
public class TimeConfig {

    /** Tiêm Clock thay vì gọi Instant.now() trực tiếp, để test có thể cố định thời gian. */
    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }
}
```

### 4.3 `mail/MailProps.java`

**`src/main/java/com/metroticket_api/mail/MailProps.java`**

```java
package com.metroticket_api.mail;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.validation.annotation.Validated;
```

```java
/**
 * Ánh xạ các khoá app.mail.* trong application.properties.
 * Nếu thiếu SMTP_FROM_ADDRESS, ứng dụng sẽ báo lỗi ngay khi khởi động.
 */
@Validated
@ConfigurationProperties(prefix = "app.mail")
public record MailProps(
        @NotBlank @Email String from,
        @NotBlank String fromName) {
}
```

`@Validated` + `@NotBlank @Email`: nếu quên đặt `SMTP_FROM_ADDRESS`, ứng dụng **báo lỗi ngay khi khởi động** thay vì đến lúc gửi mail mới lỗi.

### 4.4 `mail/MailMessage.java`

**`src/main/java/com/metroticket_api/mail/MailMessage.java`**

```java
package com.metroticket_api.mail;

import java.util.Objects;

/**
 * Một email cần gửi. Luôn có bản text (cho client không hiển thị HTML)
 * và có thể có thêm bản HTML.
 */
public record MailMessage(String to, String subject, String textBody, String htmlBody) {

    public MailMessage {
        Objects.requireNonNull(to, "to");
        Objects.requireNonNull(subject, "subject");
        Objects.requireNonNull(textBody, "textBody");
    }

    public static MailMessage text(String to, String subject, String textBody) {
        return new MailMessage(to, subject, textBody, null);
    }

    public boolean hasHtml() {
        return htmlBody != null && !htmlBody.isBlank();
    }
}
```

### 4.5 `mail/MailSender.java` (thay cho `Mailsender.java`)

**`src/main/java/com/metroticket_api/mail/MailSender.java`**

```java
package com.metroticket_api.mail;

/**
 * Cổng gửi mail của hệ thống. Code nghiệp vụ chỉ phụ thuộc interface này,
 * nhờ vậy có thể thay SMTP bằng dịch vụ khác (SES, SendGrid...) hoặc mock khi test.
 */
public interface MailSender {

    void send(MailMessage message);
}
```

> Lưu ý trùng tên: Spring cũng có `org.springframework.mail.MailSender`. Trong code dự án luôn import `com.metroticket_api.mail.MailSender`.

### 4.6 `mail/MailSendException.java`

**`src/main/java/com/metroticket_api/mail/MailSendException.java`**

```java
package com.metroticket_api.mail;

public class MailSendException extends RuntimeException {

    public MailSendException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

### 4.7 mail/SmtpMailSender.java

**src/main/java/com/metroticket_api/mail/SmtpMailSender.java**

```java
package com.metroticket_api.mail;

import java.io.UnsupportedEncodingException;
import java.nio.charset.StandardCharsets;

import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.mail.MailException;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Component;

/**
 * Gửi mail qua SMTP bằng JavaMailSender do Spring Boot tự cấu hình
 * từ các khoá spring.mail.*.
 */
@Component
public class SmtpMailSender implements MailSender {

    private static final Logger log = LoggerFactory.getLogger(SmtpMailSender.class);

    private final JavaMailSender javaMailSender;
    private final MailProps props;

    public SmtpMailSender(JavaMailSender javaMailSender, MailProps props) {
        this.javaMailSender = javaMailSender;
        this.props = props;
    }

    @Override
    public void send(MailMessage message) {
        try {
            MimeMessage mime = javaMailSender.createMimeMessage();
            // multipart = true khi có cả text và HTML
            MimeMessageHelper helper = new MimeMessageHelper(mime, message.hasHtml(),
                    StandardCharsets.UTF_8.name());
            helper.setFrom(props.from(), props.fromName());
            helper.setTo(message.to());
            helper.setSubject(message.subject());
            if (message.hasHtml()) {
                helper.setText(message.textBody(), message.htmlBody());
            } else {
                helper.setText(message.textBody(), false);
            }
            javaMailSender.send(mime);
            log.info("Đã gửi mail '{}' tới {}", message.subject(), mask(message.to()));
        } catch (MessagingException | UnsupportedEncodingException | MailException e) {
            throw new MailSendException("Không gửi được mail tới " + mask(message.to()), e);
        }
    }

    /** Che bớt địa chỉ email trong log: an***@gmail.com */
    static String mask(String email) {
        int at = email.indexOf('@');
        if (at <= 2) {
            return "***" + email.substring(Math.max(at, 0));
        }
        return email.substring(0, 2) + "***" + email.substring(at);
    }
}
```

Những điểm cần chú ý:

- `MimeMessageHelper(mime, multipart, "UTF-8")`: bắt buộc UTF-8 để tiếng Việt không lỗi font.
- `setText(text, html)`: gửi cả hai phiên bản. Client không hiển thị HTML sẽ dùng bản text.
- **Che email trong log** (`mask`) để log không chứa dữ liệu cá nhân.
- Bọc mọi lỗi vào `MailSendException` để nơi gọi chỉ cần bắt một loại exception.

### 4.8 `mail/MailTemplates.java`

**src/main/java/com/metroticket_api/mail/MailTemplates.java**

```
package com.metroticket_api.mail;

import java.time.Duration;
import java.util.Locale;

import org.springframework.stereotype.Component;
import org.thymeleaf.TemplateEngine;
import org.thymeleaf.context.Context;

/**
 * Dựng nội dung email từ template Thymeleaf trong src/main/resources/templates/mail/.
 */
@Component
public class MailTemplates {

    private static final Locale VI = Locale.forLanguageTag("vi-VN");

    private final TemplateEngine templateEngine;

    public MailTemplates(TemplateEngine templateEngine) {
        this.templateEngine = templateEngine;
    }

    public MailMessage verificationCode(String to, String code, Duration lifetime) {
        Context ctx = new Context(VI);
        ctx.setVariable("code", code);
        ctx.setVariable("minutes", lifetime.toMinutes());

        String html = templateEngine.process("mail/verification-code", ctx);
        String text = """
                Mã xác thực MetroTicket của bạn là: %s
                Mã có hiệu lực trong %d phút. Không chia sẻ mã này cho bất kỳ ai.
                """.formatted(code, lifetime.toMinutes());

        return new MailMessage(to, "[MetroTicket] Mã xác thực email", text, html);
    }
}
```

### 4.9 Template `src/main/resources/templates/mail/verification-code.html`

**src/main/resources/templates/mail/verification-code.html**

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="vi">
<head>
    <meta charset="UTF-8">
    <title>Mã xác thực MetroTicket</title>
</head>
<body style="margin:0;padding:24px;background:#f4f6f8;font-family:Arial,Helvetica,sans-serif;color:#
      1f2933;">
<table role="presentation" width="100%" cellpadding="0" cellspacing="0"
       style="max-width:480px;margin:0 auto;background:#ffffff;border-radius:8px;padding:32px;">
    <tr><td>
        <h2 style="margin:0 0 16px;color:#0b5394;">MetroTicket</h2>
        <p>Mã xác thực của bạn là:</p>
        <p style="font-size:32px;font-weight:bold;letter-spacing:8px;margin:16px 0;"
              th:text="${code}">123456</p>
        <p>Mã có hiệu lực trong <strong th:text="${minutes}">10</strong> phút.</p>
        <p style="color:#7b8794;font-size:13px;">
            Không chia sẻ mã này cho bất kỳ ai. Nếu bạn không yêu cầu, hãy bỏ qua email này.
        </p>
```

```
    </td></tr>
</table>
</body>
</html>
```

- Dùng `th:text` (không dùng `th:utext`) để Thymeleaf tự escape, tránh chèn HTML độc hại.
- CSS viết **inline** vì Gmail/Outlook bỏ qua thẻ `<style>`.

### 4.10 Thêm một loại mail mới

Ví dụ gửi mail xác nhận mua vé:

1. Tạo template `templates/mail/ticket-purchased.html`.
2. Thêm method `ticketPurchased(...)` trong `MailTemplates` trả về `MailMessage`.
3. Trong service mua vé, `publishEvent(new TicketPurchased(...))`.
4. Tạo listener `@Async(MAIL_EXECUTOR) @TransactionalEventListener(phase = AFTER_COMMIT)` gọi `mailSender.send(...)`.

---

## 5. Module Verification (mã xác thực qua email)

### 5.1 Quy tắc nghiệp vụ

| Quy tắc | Giá trị | Cấu hình |
|---|---|---|
| Độ dài mã | 6 chữ số, sinh bằng `SecureRandom` | — |
| Thời hạn | 10 phút | `app.verification.lifetime` |
| Gửi lại tối thiểu sau | 60 giây (trả `429` + header `Retry-After`) | `app.verification.resend-cooldown` |
| Số lần nhập sai tối đa | 5, sau đó phải xin mã mới | `app.verification.max-failed-attempts` |
| Lưu trữ | `HMAC-SHA256(hashKey, purpose:email:code)` | `app.verification.hash-key` |
| Mã cũ | Bị vô hiệu khi cấp mã mới | — |

**Vì sao dùng HMAC thay vì SHA-256 thường?** Mã chỉ có 1 triệu khả năng. Nếu dùng SHA-256 không khoá, ai có DB có thể thử hết trong chưa tới một giây. HMAC cần thêm `hash-key` (nằm ở biến môi trường, không nằm trong DB).

### 5.2 `VerificationProps.java` (thay cho `VerificationsProps.java`)

**`src/main/java/com/metroticket_api/verification/VerificationProps.java`**

```
package com.metroticket_api.verification;

import java.time.Duration;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.validation.annotation.Validated;

/**
 * Ánh xạ app.verification.* (hash-key, lifetime, resend-cooldown, max-failed-attempts).
 */
@Validated
@ConfigurationProperties(prefix = "app.verification")
public record VerificationProps(
        @NotNull @Size(min = 32, message = "hash-key phải dài tối thiểu 32 ký tự") String hashKey,
        @NotNull Duration lifetime,
```

```
        @NotNull Duration resendCooldown,
        @Min(1) int maxFailedAttempts) {
}
```

### 5.3 `VerificationPurpose.java` và `VerificationResult.java`

**src/main/java/com/metroticket_api/verification/VerificationPurpose.java**

```
package com.metroticket_api.verification;

public enum VerificationPurpose {
    REGISTER,
    RESET_PASSWORD
}
```

**src/main/java/com/metroticket_api/verification/VerificationResult.java**

```
package com.metroticket_api.verification;

public enum VerificationResult {
    SUCCESS,
    INVALID_CODE,
    EXPIRED,
    TOO_MANY_ATTEMPTS,
    NOT_FOUND
}
```

### 5.4 `EmailVerificationChallenge.java` (entity, thay cho file sai chính tả)

**src/main/java/com/metroticket_api/verification/EmailVerificationChallenge.java**

```
package com.metroticket_api.verification;

import java.time.Instant;
import java.util.UUID;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.persistence.Version;

/**
 * Một lần cấp mã xác thực. Chỉ lưu HMAC của mã, không bao giờ lưu mã gốc.
 */
@Entity
@Table(name = "email_verification_challenge")
public class EmailVerificationChallenge {

    @Id
    private UUID id;

    @Column(nullable = false, length = 320)
    private String email;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 32)
    private VerificationPurpose purpose;

    @Column(name = "code_hash", nullable = false, length = 64)
    private String codeHash;

    @Column(name = "expires_at", nullable = false)
    private Instant expiresAt;

    @Column(name = "failed_attempts", nullable = false)
    private int failedAttempts;

    @Column(name = "consumed_at")
```

```
    private Instant consumedAt;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    // Khoá lạc quan: chặn 2 request verify song song cùng dùng một mã
    @Version
    private long version;

    protected EmailVerificationChallenge() {
        // cho JPA
    }

    public EmailVerificationChallenge(String email, VerificationPurpose purpose, String codeHash,
                                      Instant now, Instant expiresAt) {
        this.id = UUID.randomUUID();
        this.email = email;
        this.purpose = purpose;
        this.codeHash = codeHash;
        this.createdAt = now;
        this.expiresAt = expiresAt;
    }

    public boolean isExpired(Instant now) {
        return !now.isBefore(expiresAt);
    }

    public boolean isConsumed() {
        return consumedAt != null;
    }

    public void recordFailedAttempt() {
        failedAttempts++;
    }

    public void consume(Instant now) {
        consumedAt = now;
    }

    public UUID getId() { return id; }
    public String getEmail() { return email; }
    public VerificationPurpose getPurpose() { return purpose; }
    public String getCodeHash() { return codeHash; }
    public Instant getExpiresAt() { return expiresAt; }
    public int getFailedAttempts() { return failedAttempts; }
    public Instant getCreatedAt() { return createdAt; }
}
```

`@Version` (khoá lạc quan): nếu hai request verify cùng lúc dùng chung một mã, request thứ hai sẽ lỗi `OptimisticLockException` thay vì cả hai cùng thành công.

### 5.5 `EmailVerificationChallengeRepository.java`

**src/main/java/com/metroticket_api/verification/EmailVerificationChallengeRepository.java**

```
package com.metroticket_api.verification;

import java.time.Instant;
import java.util.Optional;
import java.util.UUID;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;

public interface EmailVerificationChallengeRepository extends
        JpaRepository<EmailVerificationChallenge, UUID> {

    /** Mã mới nhất chưa dùng của một email cho một mục đích. */
    Optional<EmailVerificationChallenge>
            findFirstByEmailAndPurposeAndConsumedAtIsNullOrderByCreatedAtDesc(
            String email, VerificationPurpose purpose);

    /** Vô hiệu các mã cũ khi cấp mã mới, để chỉ mã mới nhất còn dùng được. */
    @Modifying
```

```
    @Query("""
            update EmailVerificationChallenge c set c.consumedAt = :now
            where c.email = :email and c.purpose = :purpose and c.consumedAt is null
            """)
    int invalidateActive(String email, VerificationPurpose purpose, Instant now);

    /** Dọn rác định kỳ. */
    @Modifying
    @Query("delete from EmailVerificationChallenge c where c.expiresAt < :before")
    int deleteExpiredBefore(Instant before);
}
```

### 5.6 `VerificationCodeIssued.java` (sự kiện)

**src/main/java/com/metroticket_api/verification/VerificationCodeIssued.java**

```
package com.metroticket_api.verification;

import java.time.Duration;

/**
 * Sự kiện phát ra khi một mã mới được cấp. Mã gốc chỉ tồn tại trong bộ nhớ
 * (trong sự kiện này) đủ lâu để gửi mail, không bao giờ ghi xuống DB hay log.
 */
public record VerificationCodeIssued(String email, VerificationPurpose purpose, String code,
        Duration lifetime) {

    @Override
    public String toString() {
        // tránh lộ mã nếu ai đó log cả object
        return "VerificationCodeIssued[email=" + email + ", purpose=" + purpose + "]";
    }
}
```

### 5.7 `ResendTooSoonException.java`

**src/main/java/com/metroticket_api/verification/ResendTooSoonException.java**

```
package com.metroticket_api.verification;

import java.time.Duration;

public class ResendTooSoonException extends RuntimeException {

    private final Duration retryAfter;

    public ResendTooSoonException(Duration retryAfter) {
        super("Vui lòng chờ " + retryAfter.toSeconds() + " giây trước khi yêu cầu mã mới");
        this.retryAfter = retryAfter;
    }

    public Duration getRetryAfter() {
        return retryAfter;
    }
}
```

### 5.8 `VerificationCodeService.java`

**src/main/java/com/metroticket_api/verification/VerificationCodeService.java**

```
package com.metroticket_api.verification;

import java.nio.charset.StandardCharsets;
import java.security.InvalidKeyException;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
```

```java
import java.util.HexFormat;
import java.util.Locale;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;

import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

/**
 * Cấp và kiểm tra mã xác thực 6 chữ số gửi qua email.
 *
 * <ul>
 *   <li>Mã sinh bằng SecureRandom, chỉ lưu HMAC-SHA256(hashKey, purpose:email:code).</li>
 *   <li>Giới hạn gửi lại (resend-cooldown) và số lần nhập sai (max-failed-attempts).</li>
 *   <li>Mail được gửi sau khi transaction commit (xem VerificationMailListener).</li>
 * </ul>
 */
@Service
public class VerificationCodeService {

    private static final String HMAC_ALGORITHM = "HmacSHA256";

    private final EmailVerificationChallengeRepository repository;
    private final ApplicationEventPublisher events;
    private final VerificationProps props;
    private final Clock clock;
    private final SecureRandom random = new SecureRandom();
    private final SecretKeySpec hmacKey;

    public VerificationCodeService(EmailVerificationChallengeRepository repository,
                                   ApplicationEventPublisher events,
                                   VerificationProps props,
                                   Clock clock) {
        this.repository = repository;
        this.events = events;
        this.props = props;
        this.clock = clock;
        this.hmacKey = new SecretKeySpec(props.hashKey().getBytes(StandardCharsets.UTF_8),
                HMAC_ALGORITHM);
    }

    @Transactional
    public void issue(String rawEmail, VerificationPurpose purpose) {
        String email = normalize(rawEmail);
        Instant now = clock.instant();

        repository.findFirstByEmailAndPurposeAndConsumedAtIsNullOrderByCreatedAtDesc(email, purpose)
                .ifPresent(last -> {
                    Instant allowedAt = last.getCreatedAt().plus(props.resendCooldown());
                    if (now.isBefore(allowedAt)) {
                        throw new ResendTooSoonException(Duration.between(now, allowedAt));
                    }
                });

        repository.invalidateActive(email, purpose, now);

        String code = "%06d".formatted(random.nextInt(1_000_000));
        repository.save(new EmailVerificationChallenge(
                email, purpose, hash(email, purpose, code), now, now.plus(props.lifetime())));

        events.publishEvent(new VerificationCodeIssued(email, purpose, code, props.lifetime()));
    }

    /**
     * Trả về kết quả thay vì ném exception, để số lần nhập sai vẫn được commit
     * (ném RuntimeException trong @Transactional sẽ rollback cả lần đếm sai).
     */
    @Transactional
    public VerificationResult verify(String rawEmail, VerificationPurpose purpose, String code) {
        String email = normalize(rawEmail);
        Instant now = clock.instant();

        var found =
                repository.findFirstByEmailAndPurposeAndConsumedAtIsNullOrderByCreatedAtDesc(email,
                purpose);
        if (found.isEmpty()) {
            return VerificationResult.NOT_FOUND;
```

```
        }
        EmailVerificationChallenge challenge = found.get();

        if (challenge.isExpired(now)) {
            return VerificationResult.EXPIRED;
        }
        if (challenge.getFailedAttempts() >= props.maxFailedAttempts()) {
            return VerificationResult.TOO_MANY_ATTEMPTS;
        }

        String expected = challenge.getCodeHash();
        String actual = hash(email, purpose, code == null ? "" : code.trim());
        // So sánh thời gian hằng, tránh tấn công đo thời gian (timing attack)
        boolean match = MessageDigest.isEqual(
                expected.getBytes(StandardCharsets.US_ASCII),
                        actual.getBytes(StandardCharsets.US_ASCII));

        if (!match) {
            challenge.recordFailedAttempt();
            return challenge.getFailedAttempts() >= props.maxFailedAttempts()
                    ? VerificationResult.TOO_MANY_ATTEMPTS
                    : VerificationResult.INVALID_CODE;
        }

        challenge.consume(now);
        return VerificationResult.SUCCESS;
    }

    private String hash(String email, VerificationPurpose purpose, String code) {
        try {
            Mac mac = Mac.getInstance(HMAC_ALGORITHM); // Mac không thread-safe → tạo mới mỗi lần
            mac.init(hmacKey);
            byte[] digest = mac.doFinal((purpose + ":" + email + ":" +
                    code).getBytes(StandardCharsets.UTF_8));
            return HexFormat.of().formatHex(digest);
        } catch (NoSuchAlgorithmException | InvalidKeyException e) {
            throw new IllegalStateException("Không khởi tạo được HMAC", e);
        }
    }

    static String normalize(String email) {
        return email.trim().toLowerCase(Locale.ROOT);
    }
}
```

Giải thích:

- **`MessageDigest.isEqual`**: so sánh trong thời gian không đổi. Dùng `String.equals` sẽ dừng ở ký tự sai đầu tiên, kẻ tấn công có thể đo thời gian phản hồi để đoán dần.
- **`verify()` trả về `VerificationResult`, không ném exception.** Lý do ở mục 5.11.
- **Chuẩn hoá email** (`trim` + `toLowerCase`) ở mọi nơi, nếu không `An@Gmail.com` và `an@gmail.com` sẽ thành hai tài khoản.

### 5.9 `VerificationMailListener.java`

**`src/main/java/com/metroticket_api/verification/VerificationMailListener.java`**

```
package com.metroticket_api.verification;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Component;
import org.springframework.transaction.event.TransactionPhase;
import org.springframework.transaction.event.TransactionalEventListener;

import com.metroticket_api.config.AsyncConfig;
import com.metroticket_api.mail.MailSendException;
import com.metroticket_api.mail.MailSender;
import com.metroticket_api.mail.MailTemplates;

/**
 * Nghe sự kiện VerificationCodeIssued và gửi mail.
```

```
 *
 * AFTER_COMMIT: chỉ gửi khi mã đã lưu thành công vào DB (nếu transaction rollback thì không gửi).
 * @Async: chạy trên mailExecutor, request HTTP trả về ngay không chờ SMTP.
 */
@Component
public class VerificationMailListener {

    private static final Logger log = LoggerFactory.getLogger(VerificationMailListener.class);

    private final MailSender mailSender;
    private final MailTemplates templates;

    public VerificationMailListener(MailSender mailSender, MailTemplates templates) {
        this.mailSender = mailSender;
        this.templates = templates;
    }

    @Async(AsyncConfig.MAIL_EXECUTOR)
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onCodeIssued(VerificationCodeIssued event) {
        try {
            mailSender.send(templates.verificationCode(event.email(), event.code(),
                    event.lifetime()));
        } catch (MailSendException e) {
            // Người dùng có thể bấm "gửi lại mã" sau cooldown
            log.error("Gửi mã xác thực thất bại: {}", e.getMessage(), e);
        }
    }
}
```

> **Bẫy thường gặp:** `@TransactionalEventListener` chỉ chạy khi sự kiện được publish **bên trong một transaction**. Nếu gọi `issue()` từ chỗ không có `@Transactional`, mail sẽ không bao giờ được gửi. `issue()` đã có `@Transactional` nên an toàn.

### 5.10 `VerificationCleanupJob.java`

**src/main/java/com/metroticket_api/verification/VerificationCleanupJob.java**

```
package com.metroticket_api.verification;

import java.time.Clock;
import java.time.Duration;

import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

/** Xoá các mã đã hết hạn quá 1 ngày, chạy lúc 3h sáng mỗi ngày. */
@Component
public class VerificationCleanupJob {

    private final EmailVerificationChallengeRepository repository;
    private final Clock clock;

    public VerificationCleanupJob(EmailVerificationChallengeRepository repository, Clock clock) {
        this.repository = repository;
        this.clock = clock;
    }

    @Scheduled(cron = "0 0 3 * * *", zone = "Asia/Ho_Chi_Minh")
    @Transactional
    public void purgeExpired() {
        repository.deleteExpiredBefore(clock.instant().minus(Duration.ofDays(1)));
    }
}
```

### 5.11 [!] Lỗi thực tế phát hiện khi test: rollback làm mất số lần nhập sai

Phiên bản đầu, `AuthService.verifyEmail()` dùng `@Transactional` thường. Khi mã sai, service ném `AuthException` (RuntimeException), Spring **rollback toàn bộ transaction**, kể cả việc tăng `failed_attempts`. Hậu quả: giới hạn 5 lần **không có tác dụng** và kẻ tấn công có thể thử cả 1 triệu mã.

Test `wrongCodeIsLimited` đã bắt được lỗi này (`expected 429 but was 400`). Cách sửa:

```
@Transactional(noRollbackFor = AuthException.class)
public void verifyEmail(String rawEmail, String code) { ... }
```

Nguyên tắc chung: **khi bộ đếm bảo mật phải được lưu dù thao tác thất bại, không được để exception làm rollback nó.**

---

## 6. Phân quyền (RBAC + JWT)

### 6.1 Mô hình

- Mỗi người dùng có **một vai trò** (`Role`).
- Mỗi vai trò gồm **nhiều quyền** (`Permission`), khai báo cứng trong enum.
- JWT chỉ chứa vai trò. Quyền được **suy ra khi kiểm tra**, nên đổi quyền của vai trò không cần cấp lại token.

### 6.2 Ma trận quyền

| Quyền | PASSENGER (hành khách) | STAFF (nhân viên ga) | ADMIN |
|---|---|---|---|
| `TICKET_BUY`: mua vé | [OK] | | [OK] |
| `TICKET_VIEW_OWN`: xem vé của mình | [OK] | [OK] | [OK] |
| `TICKET_VALIDATE`: soát vé | | [OK] | [OK] |
| `TICKET_VIEW_ALL`: xem mọi vé | | [OK] | [OK] |
| `STATION_MANAGE`: quản lý ga, tuyến | | | [OK] |
| `REPORT_VIEW`: xem báo cáo | | | [OK] |
| `USER_MANAGE`: quản lý tài khoản | | | [OK] |

| Đường dẫn | Ai được truy cập |
|---|---|
| `POST /api/auth/**`, `/actuator/health` | Mọi người (không cần token) |
| `/api/admin/**` | `ROLE_ADMIN` |
| `/api/staff/**` | `ROLE_STAFF`, `ROLE_ADMIN` |
| `/api/**` còn lại | Đã đăng nhập |
| Mọi đường dẫn khác | Từ chối (`denyAll`) |

### 6.3 `user/Permission.java`

**`src/main/java/com/metroticket_api/user/Permission.java`**

```java
package com.metroticket_api.user;

/**
 * Quyền chi tiết. Tên enum chính là authority mà Spring Security kiểm tra,
 * ví dụ @PreAuthorize("hasAuthority('TICKET_VALIDATE')").
 */
public enum Permission {
    TICKET_BUY,
    TICKET_VIEW_OWN,
    TICKET_VALIDATE,
    TICKET_VIEW_ALL,
    STATION_MANAGE,
    REPORT_VIEW,
    USER_MANAGE
}
```

### 6.4 user/Role.java

**src/main/java/com/metroticket_api/user/Role.java**

```java
package com.metroticket_api.user;

import static com.metroticket_api.user.Permission.*;

import java.util.Collections;
import java.util.EnumSet;
import java.util.Set;

/**
 * Vai trò của người dùng và tập quyền tương ứng.
 * Muốn đổi quyền của một vai trò chỉ cần sửa ở đây, không cần đổi DB hay token.
 */
public enum Role {
    PASSENGER(EnumSet.of(TICKET_BUY, TICKET_VIEW_OWN)),
    STAFF(EnumSet.of(TICKET_VIEW_OWN, TICKET_VALIDATE, TICKET_VIEW_ALL)),
    ADMIN(EnumSet.allOf(Permission.class));

    private final Set<Permission> permissions;

    Role(Set<Permission> permissions) {
        this.permissions = Collections.unmodifiableSet(permissions);
    }

    public Set<Permission> permissions() {
        return permissions;
    }

    /** Authority dạng ROLE_xxx dùng cho hasRole('xxx'). */
    public String authority() {
        return "ROLE_" + name();
    }
}
```

### 6.5 user/UserStatus.java

**src/main/java/com/metroticket_api/user/UserStatus.java**

```java
package com.metroticket_api.user;

public enum UserStatus {
    PENDING_VERIFICATION,
    ACTIVE,
    LOCKED
}
```

### 6.6 user/User.java

**src/main/java/com/metroticket_api/user/User.java**

```java
package com.metroticket_api.user;

import java.time.Instant;
import java.util.UUID;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "app_user") // "user" là từ khoá trong PostgreSQL
public class User {

    @Id
    private UUID id;

    @Column(nullable = false, unique = true, length = 320)
    private String email;

    @Column(name = "password_hash", nullable = false)
    private String passwordHash;

    @Column(name = "full_name", nullable = false, length = 120)
    private String fullName;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 32)
    private Role role;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 32)
    private UserStatus status;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    protected User() {
        // cho JPA
    }

    public User(String email, String passwordHash, String fullName, Role role, UserStatus status,
            Instant now) {
        this.id = UUID.randomUUID();
        this.email = email;
        this.passwordHash = passwordHash;
        this.fullName = fullName;
        this.role = role;
        this.status = status;
        this.createdAt = now;
    }

    public void activate() {
        if (status == UserStatus.PENDING_VERIFICATION) {
            status = UserStatus.ACTIVE;
        }
    }

    public void changePassword(String newHash) { this.passwordHash = newHash; }
    public void changeRole(Role role) { this.role = role; }
    public void changeStatus(UserStatus status) { this.status = status; }

    public UUID getId() { return id; }
    public String getEmail() { return email; }
    public String getPasswordHash() { return passwordHash; }
    public String getFullName() { return fullName; }
    public Role getRole() { return role; }
    public UserStatus getStatus() { return status; }
    public Instant getCreatedAt() { return createdAt; }
}
```

### 6.7 user/UserRepository.java

**src/main/java/com/metroticket_api/user/UserRepository.java**

```java
package com.metroticket_api.user;

import java.util.Optional;
import java.util.UUID;

import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, UUID> {

    Optional<User> findByEmail(String email);

    boolean existsByEmail(String email);

    boolean existsByRole(Role role);
}
```

### 6.8 security/JwtProps.java

**src/main/java/com/metroticket_api/security/JwtProps.java**

```java
package com.metroticket_api.security;

import java.time.Duration;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.validation.annotation.Validated;

/** app.jwt.secret (>= 32 byte cho HS256), app.jwt.issuer, app.jwt.access-token-ttl */
@Validated
@ConfigurationProperties(prefix = "app.jwt")
public record JwtProps(
        @NotNull @Size(min = 32, message = "app.jwt.secret phải dài tối thiểu 32 ký tự") String
                secret,
        @NotBlank String issuer,
        @NotNull Duration accessTokenTtl) {
}
```

### 6.9 security/JwtConfig.java

**src/main/java/com/metroticket_api/security/JwtConfig.java**

```java
package com.metroticket_api.security;

import java.nio.charset.StandardCharsets;

import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.core.DelegatingOAuth2TokenValidator;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.JwtValidators;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusJwtEncoder;

import com.nimbusds.jose.jwk.source.ImmutableSecret;

/**
 * Hệ thống tự phát hành JWT ký HS256 và tự kiểm tra (oauth2-resource-server).
 */
@Configuration
public class JwtConfig {

    private final SecretKey key;
    private final JwtProps props;
```

```
    public JwtConfig(JwtProps props) {
        this.props = props;
        this.key = new SecretKeySpec(props.secret().getBytes(StandardCharsets.UTF_8), "HmacSHA256");
    }

    @Bean
    public JwtEncoder jwtEncoder() {
        return new NimbusJwtEncoder(new ImmutableSecret<>(key));
    }

    @Bean
    public JwtDecoder jwtDecoder() {
        NimbusJwtDecoder decoder = NimbusJwtDecoder.withSecretKey(key)
                .macAlgorithm(MacAlgorithm.HS256)
                .build();
        // Kiểm tra hạn dùng (exp/nbf) và issuer
        decoder.setJwtValidator(new DelegatingOAuth2TokenValidator<>(
                JwtValidators.createDefaultWithIssuer(props.issuer())));
        return decoder;
    }
}
```

Dự án đã có `spring-boot-starter-security-oauth2-resource-server` nên dùng luôn `NimbusJwtEncoder`/`NimbusJwtDecoder`, không cần thêm thư viện JWT nào khác (jjwt...).

### 6.10 `security/TokenService.java`

**`src/main/java/com/metroticket_api/security/TokenService.java`**

```
package com.metroticket_api.security;

import java.time.Clock;
import java.time.Instant;
import java.util.List;

import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.JwsHeader;
import org.springframework.security.oauth2.jwt.JwtClaimsSet;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.JwtEncoderParameters;
import org.springframework.stereotype.Service;

import com.metroticket_api.user.User;

@Service
public class TokenService {

    public static final String ROLES_CLAIM = "roles";

    private final JwtEncoder encoder;
    private final JwtProps props;
    private final Clock clock;

    public TokenService(JwtEncoder encoder, JwtProps props, Clock clock) {
        this.encoder = encoder;
        this.props = props;
        this.clock = clock;
    }

    public IssuedToken issueAccessToken(User user) {
        Instant now = clock.instant();
        Instant expiresAt = now.plus(props.accessTokenTtl());
        JwtClaimsSet claims = JwtClaimsSet.builder()
                .issuer(props.issuer())
                .subject(user.getId().toString())
                .issuedAt(now)
                .expiresAt(expiresAt)
                .claim("email", user.getEmail())
                // Chỉ đưa vai trò vào token; quyền chi tiết được suy ra từ Role khi kiểm tra
                .claim(ROLES_CLAIM, List.of(user.getRole().name()))
                .build();
        JwsHeader header = JwsHeader.with(MacAlgorithm.HS256).build();
        String value = encoder.encode(JwtEncoderParameters.from(header, claims)).getTokenValue();
        return new IssuedToken(value, expiresAt);
    }
```

```
    public record IssuedToken(String value, Instant expiresAt) {
    }
}
```

### 6.11 security/RoleAuthoritiesConverter.java

**src/main/java/com/metroticket_api/security/RoleAuthoritiesConverter.java**

```
package com.metroticket_api.security;

import java.util.ArrayList;
import java.util.Collection;
import java.util.List;

import org.springframework.core.convert.converter.Converter;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.oauth2.jwt.Jwt;

import com.metroticket_api.user.Role;

/**
 * Đổi claim "roles" trong JWT thành danh sách authority:
 * roles=["STAFF"] → ROLE_STAFF, TICKET_VIEW_OWN, TICKET_VALIDATE, TICKET_VIEW_ALL
 */
public class RoleAuthoritiesConverter implements Converter<Jwt, Collection<GrantedAuthority>> {

    @Override
    public Collection<GrantedAuthority> convert(Jwt jwt) {
        List<String> roles = jwt.getClaimAsStringList(TokenService.ROLES_CLAIM);
        if (roles == null) {
            return List.of();
        }
        List<GrantedAuthority> authorities = new ArrayList<>();
        for (String name : roles) {
            Role role;
            try {
                role = Role.valueOf(name);
            } catch (IllegalArgumentException unknown) {
                continue; // bỏ qua vai trò lạ thay vì làm hỏng cả request
            }
            authorities.add(new SimpleGrantedAuthority(role.authority()));
            role.permissions().forEach(p -> authorities.add(new SimpleGrantedAuthority(p.name())));
        }
        return authorities;
    }
}
```

### 6.12 security/SecurityConfig.java

**src/main/java/com/metroticket_api/security/SecurityConfig.java**

```
package com.metroticket_api.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationConverter
        ;
import org.springframework.security.oauth2.server.resource.web.BearerTokenAuthenticationEntryPoint;
import org.springframework.security.oauth2.server.resource.web.access.BearerTokenAccessDeniedHandler
        ;
import org.springframework.security.web.SecurityFilterChain;

/**
 * Phân quyền 2 lớp:
```

```
 *  1. Theo URL (ở đây) — chặn thô theo nhóm đường dẫn.
 *  2. Theo method (@PreAuthorize) — kiểm tra quyền chi tiết / quyền sở hữu dữ liệu.
 */
@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
        http
            // API stateless dùng Bearer token, không dùng cookie session → không cần CSRF
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health", "/actuator/health/**").permitAll()
                .requestMatchers(HttpMethod.POST, "/api/auth/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/staff/**").hasAnyRole("STAFF", "ADMIN")
                .requestMatchers("/api/**").authenticated()
                .anyRequest().denyAll())
            .oauth2ResourceServer(o -> o.jwt(jwt ->
                    jwt.jwtAuthenticationConverter(jwtAuthenticationConverter())))
            // 401 khi thiếu/sai token, 403 khi đủ token nhưng thiếu quyền
            .exceptionHandling(e -> e
                .authenticationEntryPoint(new BearerTokenAuthenticationEntryPoint())
                .accessDeniedHandler(new BearerTokenAccessDeniedHandler()));
        return http.build();
    }

    @Bean
    public JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(new RoleAuthoritiesConverter());
        return converter;
    }

    /** Mặc định băm bằng BCrypt, mật khẩu lưu dạng "{bcrypt}$2a$10$...". */
    @Bean
    public PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }
}
```

- **`STATELESS` + tắt CSRF**: chỉ đúng khi client gửi token qua header `Authorization`. Nếu sau này lưu token trong cookie thì **phải bật lại CSRF**.
- **`anyRequest().denyAll()`**: endpoint quên cấu hình sẽ bị chặn mặc định (an toàn hơn `permitAll`).
- `@EnableMethodSecurity` bật `@PreAuthorize` cho lớp kiểm tra thứ hai.

### 6.13 `security/CurrentUser.java`

**`src/main/java/com/metroticket_api/security/CurrentUser.java`**

```
package com.metroticket_api.security;

import java.util.UUID;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;

/** Tiện ích lấy id người dùng hiện tại (claim "sub" của JWT). */
public final class CurrentUser {

    private CurrentUser() {
    }

    public static UUID id() {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        if (auth == null || !auth.isAuthenticated()) {
            throw new IllegalStateException("Chưa đăng nhập");
        }
        return UUID.fromString(auth.getName());
    }
}
```

## 6.14 `security/AdminBootstrap.java`: tạo admin đầu tiên

**`src/main/java/com/metroticket_api/security/AdminBootstrap.java`**

```java
package com.metroticket_api.security;

import java.time.Clock;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import com.metroticket_api.user.Role;
import com.metroticket_api.user.User;
import com.metroticket_api.user.UserRepository;
import com.metroticket_api.user.UserStatus;

/**
 * Tạo tài khoản ADMIN đầu tiên khi hệ thống chưa có admin nào.
 * Đặt ADMIN_EMAIL / ADMIN_PASSWORD trong .env; để trống thì bỏ qua.
 */
@Component
public class AdminBootstrap implements ApplicationRunner {

    private static final Logger log = LoggerFactory.getLogger(AdminBootstrap.class);

    private final UserRepository users;
    private final PasswordEncoder passwordEncoder;
    private final Clock clock;
    private final String email;
    private final String password;

    public AdminBootstrap(UserRepository users, PasswordEncoder passwordEncoder, Clock clock,
                          @Value("${app.bootstrap.admin-email:}") String email,
                          @Value("${app.bootstrap.admin-password:}") String password) {
        this.users = users;
        this.passwordEncoder = passwordEncoder;
        this.clock = clock;
        this.email = email;
        this.password = password;
    }

    @Override
    @Transactional
    public void run(ApplicationArguments args) {
        if (email.isBlank() || password.isBlank() || users.existsByRole(Role.ADMIN)) {
            return;
        }
        users.save(new User(email.trim().toLowerCase(), passwordEncoder.encode(password),
                "Quản trị viên", Role.ADMIN, UserStatus.ACTIVE, clock.instant()));
        log.warn("Đã tạo tài khoản ADMIN đầu tiên: {}. Hãy đổi mật khẩu ngay.", email);
    }
}
```

Không đặt admin trong migration SQL, vì mật khẩu cần được băm bằng BCrypt và không được nằm trong mã nguồn.

## 6.15 Cách dùng `@PreAuthorize` trong các controller sau này

```java
// Chỉ người có quyền soát vé
@PreAuthorize("hasAuthority('TICKET_VALIDATE')")
@PostMapping("/api/staff/tickets/{code}/validate")
public ValidationResult validate(@PathVariable String code) { ... }

// Quyền sở hữu: hành khách chỉ xem vé của chính mình, nhân viên/admin xem tất cả
@PreAuthorize("#userId.toString() == authentication.name or hasAuthority('TICKET_VIEW_ALL')")
@GetMapping("/api/users/{userId}/tickets")
public List<TicketView> tickets(@PathVariable UUID userId) { ... }
```

> Ưu tiên `hasAuthority('QUYEN')` hơn `hasRole('VAI_TRO')` trong method. Khi thêm vai trò mới (ví dụ `INSPECTOR`), chỉ cần sửa `Role.java`.

---

# 7. API đăng ký, đăng nhập, quản trị

## 7.1 Danh sách endpoint

| Method | Endpoint | Quyền | Body | Kết quả |
|---|---|---|---|---|
| POST | `/api/auth/register` | công khai | `email, password, fullName` | `202`; `409` nếu email đã có |
| POST | `/api/auth/verify-email` | công khai | `email, code` | `204`; `400` sai/hết hạn; `429` sai quá 5 lần |
| POST | `/api/auth/resend-code` | công khai | `email` | `202`; `429` nếu chưa hết cooldown |
| POST | `/api/auth/login` | công khai | `email, password` | `200 {accessToken, tokenType, expiresAt}`; `401`; `403` chưa xác thực/bị khoá |
| POST | `/api/auth/forgot-password` | công khai | `email` | luôn `202` (chống dò tài khoản) |
| POST | `/api/auth/reset-password` | công khai | `email, code, newPassword` | `204` |
| GET | `/api/me` | đã đăng nhập | — | thông tin bản thân |
| GET | `/api/admin/users?page=0&size=20` | `USER_MANAGE` | — | danh sách phân trang |
| PATCH | `/api/admin/users/{id}/role` | `USER_MANAGE` | `{"role":"STAFF"}` | user sau khi đổi |
| PATCH | `/api/admin/users/{id}/status` | `USER_MANAGE` | `{"status":"LOCKED"}` | user sau khi đổi |

## 7.2 `auth/AuthDtos.java`

**`src/main/java/com/metroticket_api/auth/AuthDtos.java`**

```java
package com.metroticket_api.auth;

import java.time.Instant;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

/** Các request/response của /api/auth. */
public final class AuthDtos {

    private AuthDtos() {
    }

    public record RegisterRequest(
            @NotBlank @Email @Size(max = 320) String email,
```

```
            @NotBlank @Size(min = 8, max = 72, message = "Mật khẩu từ 8 đến 72 ký tự") String
                    password,
            @NotBlank @Size(max = 120) String fullName) {
    }

    public record VerifyEmailRequest(
            @NotBlank @Email String email,
            @NotBlank @Pattern(regexp = "\\d{6}", message = "Mã gồm 6 chữ số") String code) {
    }

    public record EmailRequest(@NotBlank @Email String email) {
    }

    public record LoginRequest(@NotBlank @Email String email, @NotBlank String password) {
    }

    public record ResetPasswordRequest(
            @NotBlank @Email String email,
            @NotBlank @Pattern(regexp = "\\d{6}", message = "Mã gồm 6 chữ số") String code,
            @NotBlank @Size(min = 8, max = 72) String newPassword) {
    }

    public record TokenResponse(String accessToken, String tokenType, Instant expiresAt) {
    }
}
```

> Mật khẩu tối đa **72 ký tự** vì BCrypt chỉ dùng 72 byte đầu.

### 7.3 auth/AuthException.java

**src/main/java/com/metroticket_api/auth/AuthException.java**

```
package com.metroticket_api.auth;

import org.springframework.http.HttpStatus;

public class AuthException extends RuntimeException {

    private final HttpStatus status;

    public AuthException(HttpStatus status, String message) {
        super(message);
        this.status = status;
    }

    public HttpStatus getStatus() {
        return status;
    }
}
```

### 7.4 auth/AuthService.java

**src/main/java/com/metroticket_api/auth/AuthService.java**

```
package com.metroticket_api.auth;

import java.time.Clock;

import org.springframework.http.HttpStatus;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.metroticket_api.auth.AuthDtos.TokenResponse;
import com.metroticket_api.security.TokenService;
import com.metroticket_api.user.Role;
import com.metroticket_api.user.User;
import com.metroticket_api.user.UserRepository;
import com.metroticket_api.user.UserStatus;
import com.metroticket_api.verification.VerificationCodeService;
```

```java
import com.metroticket_api.verification.VerificationPurpose;
import com.metroticket_api.verification.VerificationResult;

@Service
public class AuthService {

    private static final String BAD_CREDENTIALS = "Email hoặc mật khẩu không đúng";

    private final UserRepository users;
    private final PasswordEncoder passwordEncoder;
    private final VerificationCodeService verification;
    private final TokenService tokens;
    private final Clock clock;

    public AuthService(UserRepository users, PasswordEncoder passwordEncoder,
                       VerificationCodeService verification, TokenService tokens, Clock clock) {
        this.users = users;
        this.passwordEncoder = passwordEncoder;
        this.verification = verification;
        this.tokens = tokens;
        this.clock = clock;
    }

    /** Đăng ký: tạo user PENDING_VERIFICATION (vai trò PASSENGER) và gửi mã qua email. */
    @Transactional
    public void register(String rawEmail, String password, String fullName) {
        String email = normalize(rawEmail);
        if (users.existsByEmail(email)) {
            throw new AuthException(HttpStatus.CONFLICT, "Email đã được sử dụng");
        }
        users.save(new User(email, passwordEncoder.encode(password), fullName.trim(),
                Role.PASSENGER, UserStatus.PENDING_VERIFICATION, clock.instant()));
        verification.issue(email, VerificationPurpose.REGISTER);
    }

    @Transactional
    public void resendRegisterCode(String rawEmail) {
        String email = normalize(rawEmail);
        users.findByEmail(email)
                .filter(u -> u.getStatus() == UserStatus.PENDING_VERIFICATION)
                .ifPresent(u -> verification.issue(email, VerificationPurpose.REGISTER));
        // Không báo "email không tồn tại" để tránh dò tài khoản
    }

    /**
     * noRollbackFor: khi mã sai ta ném AuthException, nhưng vẫn phải commit
     * failed_attempts đã tăng — nếu rollback thì kẻ tấn công thử mã vô hạn lần.
     */
    @Transactional(noRollbackFor = AuthException.class)
    public void verifyEmail(String rawEmail, String code) {
        String email = normalize(rawEmail);
        VerificationResult result = verification.verify(email, VerificationPurpose.REGISTER, code);
        if (result != VerificationResult.SUCCESS) {
            throw toException(result);
        }
        users.findByEmail(email).ifPresent(User::activate);
    }

    @Transactional(readOnly = true)
    public TokenResponse login(String rawEmail, String password) {
        User user = users.findByEmail(normalize(rawEmail))
                .orElseThrow(() -> new AuthException(HttpStatus.UNAUTHORIZED, BAD_CREDENTIALS));
        if (!passwordEncoder.matches(password, user.getPasswordHash())) {
            throw new AuthException(HttpStatus.UNAUTHORIZED, BAD_CREDENTIALS);
        }
        switch (user.getStatus()) {
            case PENDING_VERIFICATION -> throw new AuthException(HttpStatus.FORBIDDEN, "Email chưa
                    được xác thực");
            case LOCKED -> throw new AuthException(HttpStatus.FORBIDDEN, "Tài khoản đã bị khoá");
            case ACTIVE -> { }
        }
        TokenService.IssuedToken token = tokens.issueAccessToken(user);
        return new TokenResponse(token.value(), "Bearer", token.expiresAt());
    }

    @Transactional
    public void forgotPassword(String rawEmail) {
        String email = normalize(rawEmail);
        users.findByEmail(email)
```

```
                .filter(u -> u.getStatus() == UserStatus.ACTIVE)
                .ifPresent(u -> verification.issue(email, VerificationPurpose.RESET_PASSWORD));
    }

    @Transactional(noRollbackFor = AuthException.class)
    public void resetPassword(String rawEmail, String code, String newPassword) {
        String email = normalize(rawEmail);
        VerificationResult result = verification.verify(email, VerificationPurpose.RESET_PASSWORD,
                code);
        if (result != VerificationResult.SUCCESS) {
            throw toException(result);
        }
        users.findByEmail(email).ifPresent(u ->
                u.changePassword(passwordEncoder.encode(newPassword)));
    }

    private static AuthException toException(VerificationResult result) {
        return switch (result) {
            case EXPIRED -> new AuthException(HttpStatus.BAD_REQUEST, "Mã đã hết hạn, vui lòng yêu
                    cầu mã mới");
            case TOO_MANY_ATTEMPTS -> new AuthException(HttpStatus.TOO_MANY_REQUESTS, "Nhập sai quá
                    nhiều lần, vui lòng yêu cầu mã mới");
            case INVALID_CODE, NOT_FOUND -> new AuthException(HttpStatus.BAD_REQUEST, "Mã xác thực
                    không đúng");
            case SUCCESS -> throw new IllegalArgumentException("SUCCESS không phải lỗi");
        };
    }

    private static String normalize(String email) {
        return email.trim().toLowerCase(java.util.Locale.ROOT);
    }
}
```

### 7.5 auth/AuthController.java

**src/main/java/com/metroticket_api/auth/AuthController.java**

```
package com.metroticket_api.auth;

import jakarta.validation.Valid;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import com.metroticket_api.auth.AuthDtos.EmailRequest;
import com.metroticket_api.auth.AuthDtos.LoginRequest;
import com.metroticket_api.auth.AuthDtos.RegisterRequest;
import com.metroticket_api.auth.AuthDtos.ResetPasswordRequest;
import com.metroticket_api.auth.AuthDtos.TokenResponse;
import com.metroticket_api.auth.AuthDtos.VerifyEmailRequest;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/register")
    @ResponseStatus(HttpStatus.ACCEPTED)
    public void register(@Valid @RequestBody RegisterRequest req) {
        authService.register(req.email(), req.password(), req.fullName());
    }

    @PostMapping("/verify-email")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void verifyEmail(@Valid @RequestBody VerifyEmailRequest req) {
        authService.verifyEmail(req.email(), req.code());
    }
```

```
    @PostMapping("/resend-code")
    @ResponseStatus(HttpStatus.ACCEPTED)
    public void resendCode(@Valid @RequestBody EmailRequest req) {
        authService.resendRegisterCode(req.email());
    }

    @PostMapping("/login")
    public TokenResponse login(@Valid @RequestBody LoginRequest req) {
        return authService.login(req.email(), req.password());
    }

    @PostMapping("/forgot-password")
    @ResponseStatus(HttpStatus.ACCEPTED)
    public void forgotPassword(@Valid @RequestBody EmailRequest req) {
        authService.forgotPassword(req.email());
    }

    @PostMapping("/reset-password")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void resetPassword(@Valid @RequestBody ResetPasswordRequest req) {
        authService.resetPassword(req.email(), req.code(), req.newPassword());
    }
}
```

### 7.6 user/UserDtos.java, MeController.java, AdminUserController.java

**src/main/java/com/metroticket_api/user/UserDtos.java**

```
package com.metroticket_api.user;

import java.util.UUID;

import jakarta.validation.constraints.NotNull;

public final class UserDtos {

    private UserDtos() {
    }

    public record UserView(UUID id, String email, String fullName, Role role, UserStatus status) {
        public static UserView of(User u) {
            return new UserView(u.getId(), u.getEmail(), u.getFullName(), u.getRole(),
                    u.getStatus());
        }
    }

    public record ChangeRoleRequest(@NotNull Role role) {
    }

    public record ChangeStatusRequest(@NotNull UserStatus status) {
    }
}
```

**src/main/java/com/metroticket_api/user/MeController.java**

```
package com.metroticket_api.user;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

import com.metroticket_api.security.CurrentUser;
import com.metroticket_api.user.UserDtos.UserView;

/** Bất kỳ người dùng đã đăng nhập nào cũng xem được thông tin của chính mình. */
@RestController
@RequestMapping("/api/me")
public class MeController {

    private final UserRepository users;
```

```
    public MeController(UserRepository users) {
        this.users = users;
    }

    @GetMapping
    public UserView me() {
        return users.findById(CurrentUser.id())
                .map(UserView::of)
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
    }
}
```

**src/main/java/com/metroticket_api/user/AdminUserController.java**

```
package com.metroticket_api.user;

import java.util.UUID;

import jakarta.validation.Valid;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.http.HttpStatus;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

import com.metroticket_api.security.CurrentUser;
import com.metroticket_api.user.UserDtos.ChangeRoleRequest;
import com.metroticket_api.user.UserDtos.ChangeStatusRequest;
import com.metroticket_api.user.UserDtos.UserView;

/** URL /api/admin/** đã yêu cầu ROLE_ADMIN; @PreAuthorize kiểm tra thêm quyền chi tiết. */
@RestController
@RequestMapping("/api/admin/users")
@PreAuthorize("hasAuthority('USER_MANAGE')")
public class AdminUserController {

    private final UserRepository users;

    public AdminUserController(UserRepository users) {
        this.users = users;
    }

    @GetMapping
    public Page<UserView> list(Pageable pageable) {
        return users.findAll(pageable).map(UserView::of);
    }

    @PatchMapping("/{id}/role")
    @Transactional
    public UserView changeRole(@PathVariable UUID id, @Valid @RequestBody ChangeRoleRequest req) {
        User user = load(id);
        if (user.getId().equals(CurrentUser.id())) {
            // Tránh admin tự hạ quyền và khoá mất quyền quản trị
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Không thể tự đổi vai trò của
                    chính mình");
        }
        user.changeRole(req.role());
        return UserView.of(user);
    }

    @PatchMapping("/{id}/status")
    @Transactional
    public UserView changeStatus(@PathVariable UUID id, @Valid @RequestBody ChangeStatusRequest
            req) {
        User user = load(id);
        user.changeStatus(req.status());
        return UserView.of(user);
    }
```

```
    private User load(UUID id) {
        return users.findById(id).orElseThrow(() -> new
                ResponseStatusException(HttpStatus.NOT_FOUND));
    }
}
```

### 7.7 config/ApiExceptionHandler.java

**src/main/java/com/metroticket_api/config/ApiExceptionHandler.java**

```
package com.metroticket_api.config;

import java.util.Map;
import java.util.stream.Collectors;

import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import com.metroticket_api.auth.AuthException;
import com.metroticket_api.verification.ResendTooSoonException;

/** Trả lỗi theo chuẩn RFC 9457 (application/problem+json). */
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(AuthException.class)
    public ProblemDetail handleAuth(AuthException e) {
        return ProblemDetail.forStatusAndDetail(e.getStatus(), e.getMessage());
    }

    @ExceptionHandler(ResendTooSoonException.class)
    public ResponseEntity<ProblemDetail> handleResend(ResendTooSoonException e) {
        ProblemDetail body = ProblemDetail.forStatusAndDetail(HttpStatus.TOO_MANY_REQUESTS,
                e.getMessage());
        return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS)
                .header(HttpHeaders.RETRY_AFTER, String.valueOf(Math.max(1,
                        e.getRetryAfter().toSeconds())))
                .body(body);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleValidation(MethodArgumentNotValidException e) {
        ProblemDetail body = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Dữ liệu
                không hợp lệ");
        Map<String, String> errors = e.getBindingResult().getFieldErrors().stream()
                .collect(Collectors.toMap(f -> f.getField(), f ->
                        String.valueOf(f.getDefaultMessage()), (a, b) -> a));
        body.setProperty("errors", errors);
        return body;
    }
}
```

## 7.8 Thử bằng curl

```
# Đăng ký
curl -i -X POST localhost:8080/api/auth/register -H 'Content-Type: application/json' \
  -d '{"email":"an@example.com","password":"MatKhau@123","fullName":"Nguyễn An"}'

# Lấy mã trong mail (Mailpit: http://localhost:8025) rồi xác thực
curl -i -X POST localhost:8080/api/auth/verify-email -H 'Content-Type: application/json' \
  -d '{"email":"an@example.com","code":"123456"}'

# Đăng nhập, lấy token
TOKEN=$(curl -s -X POST localhost:8080/api/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"an@example.com","password":"MatKhau@123"}' | jq -r .accessToken)

curl -H "Authorization: Bearer $TOKEN" localhost:8080/api/me          # 200
curl -i -H "Authorization: Bearer $TOKEN" localhost:8080/api/admin/users  # 403
```

Ví dụ phản hồi lỗi (RFC 9457):

```
{
  "type": "about:blank",
  "title": "Too Many Requests",
  "status": 429,
  "detail": "Vui lòng chờ 42 giây trước khi yêu cầu mã mới",
  "instance": "/api/auth/resend-code"
}
```

---

# 8. Database migration (Flyway)

Đặt trong `src/main/resources/db/migration/`. **Không bao giờ sửa file migration đã chạy trên môi trường thật**, mà tạo file `V3__...` mới.

**V1__create_app_user.sql**

**src/main/resources/db/migration/V1__create_app_user.sql**

```
CREATE TABLE app_user (
    id            UUID         PRIMARY KEY,
    email         VARCHAR(320) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name     VARCHAR(120) NOT NULL,
    role          VARCHAR(32)  NOT NULL CHECK (role IN ('PASSENGER', 'STAFF', 'ADMIN')),
    status        VARCHAR(32)  NOT NULL CHECK (status IN ('PENDING_VERIFICATION', 'ACTIVE',
            'LOCKED')),
    created_at    TIMESTAMPTZ  NOT NULL
);

CREATE INDEX idx_app_user_role ON app_user (role);
```

**V2__create_email_verification_challenge.sql**

**src/main/resources/db/migration/V2__create_email_verification_challenge.sql**

```
CREATE TABLE email_verification_challenge (
    id              UUID         PRIMARY KEY,
    email           VARCHAR(320) NOT NULL,
    purpose         VARCHAR(32)  NOT NULL CHECK (purpose IN ('REGISTER', 'RESET_PASSWORD')),
    code_hash       VARCHAR(64)  NOT NULL,
    expires_at      TIMESTAMPTZ  NOT NULL,
    failed_attempts INT          NOT NULL DEFAULT 0,
    consumed_at     TIMESTAMPTZ,
    created_at      TIMESTAMPTZ  NOT NULL,
    version         BIGINT       NOT NULL DEFAULT 0
);

CREATE INDEX idx_evc_lookup ON email_verification_challenge (email, purpose, created_at DESC);
CREATE INDEX idx_evc_expires ON email_verification_challenge (expires_at);
```

---

# 9. Kiểm thử

Cần **Docker đang chạy** (Testcontainers sẽ tự bật PostgreSQL). GreenMail chạy SMTP giả trong JVM ở cổng 3025.

## 9.1 src/test/resources/application-test.properties

**src/test/resources/application-test.properties**

```
# GreenMail chạy SMTP giả ở cổng 3025, không TLS
spring.mail.host=localhost
spring.mail.port=3025
spring.mail.username=test
spring.mail.password=test
spring.mail.properties.mail.smtp.starttls.enable=false
spring.mail.properties.mail.smtp.starttls.required=false
app.mail.from=no-reply@metroticket.test
app.verification.hash-key=test-hash-key-0123456789-abcdefghijklmnop
app.jwt.secret=test-jwt-secret-0123456789-abcdefghijklmnop
management.health.redis.enabled=false
management.health.mail.enabled=false
```

### 9.2 `TestcontainersConfiguration.java`

**`src/test/java/com/metroticket_api/TestcontainersConfiguration.java`**

```
package com.metroticket_api;

import org.springframework.boot.test.context.TestConfiguration;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.springframework.context.annotation.Bean;
import org.testcontainers.postgresql.PostgreSQLContainer;

/** PostgreSQL thật chạy trong Docker; @ServiceConnection tự điền spring.datasource.* */
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    @Bean
    @ServiceConnection
    PostgreSQLContainer postgres() {
        return new PostgreSQLContainer("postgres:17-alpine");
    }
}
```

### 9.3 `MetroticketApiApplicationTests.java` (sửa lại)

**`src/test/java/com/metroticket_api/MetroticketApiApplicationTests.java`**

```
package com.metroticket_api;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.test.context.ActiveProfiles;

@SpringBootTest
@ActiveProfiles("test")
@Import(TestcontainersConfiguration.class)
class MetroticketApiApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

### 9.4 `auth/AuthFlowIntegrationTest.java`: luồng đầy đủ + mail thật

**`src/test/java/com/metroticket_api/auth/AuthFlowIntegrationTest.java`**

```
package com.metroticket_api.auth;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
```

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

import jakarta.mail.internet.MimeMessage;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.RegisterExtension;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.web.servlet.MockMvc;

import com.icegreen.greenmail.configuration.GreenMailConfiguration;
import com.icegreen.greenmail.junit5.GreenMailExtension;
import com.icegreen.greenmail.util.GreenMailUtil;
import com.icegreen.greenmail.util.ServerSetupTest;
import com.jayway.jsonpath.JsonPath;
import com.metroticket_api.TestcontainersConfiguration;

/** Luồng đầy đủ: đăng ký → nhận mail → xác thực → đăng nhập → gọi API theo quyền. */
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
@Import(TestcontainersConfiguration.class)
class AuthFlowIntegrationTest {

    @RegisterExtension
    static GreenMailExtension greenMail = new GreenMailExtension(ServerSetupTest.SMTP)
            .withConfiguration(GreenMailConfiguration.aConfig().withUser("test", "test"));

    private static final Pattern CODE = Pattern.compile("\\b(\\d{6})\\b");

    @Autowired
    MockMvc mvc;

    @Test
    void registerVerifyLoginAndAccessByRole() throws Exception {
        String email = "an.nguyen@example.com";

        mvc.perform(post("/api/auth/register").contentType(MediaType.APPLICATION_JSON)
                        .content("""
                                {"email":"%s","password":"MatKhau@123","fullName":"Nguyễn An"}
                                """.formatted(email)))
                .andExpect(status().isAccepted());

        // Mail gửi bất đồng bộ sau commit → chờ tối đa 5 giây
        assertThat(greenMail.waitForIncomingEmail(5000, 1)).isTrue();
        MimeMessage mail = greenMail.getReceivedMessages()[0];
        assertThat(mail.getAllRecipients()[0].toString()).isEqualTo(email);
        assertThat(mail.getSubject()).contains("Mã xác thực");
        String code = extractCode(GreenMailUtil.getBody(mail));

        // Chưa xác thực thì không đăng nhập được
        mvc.perform(post("/api/auth/login").contentType(MediaType.APPLICATION_JSON)
                        .content(loginJson(email)))
                .andExpect(status().isForbidden());

        // Gửi lại ngay lập tức → 429 do cooldown
        mvc.perform(post("/api/auth/resend-code").contentType(MediaType.APPLICATION_JSON)
                        .content("{\"email\":\"%s\"}".formatted(email)))
                .andExpect(status().isTooManyRequests())
                .andExpect(header().exists("Retry-After"));

        mvc.perform(post("/api/auth/verify-email").contentType(MediaType.APPLICATION_JSON)
                        .content("{\"email\":\"%s\",\"code\":\"%s\"}".formatted(email, code)))
                .andExpect(status().isNoContent());

        String body = mvc.perform(post("/api/auth/login").contentType(MediaType.APPLICATION_JSON)
                        .content(loginJson(email)))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.tokenType").value("Bearer"))
                .andReturn().getResponse().getContentAsString();
        String token = JsonPath.read(body, "$.accessToken");

        mvc.perform(get("/api/me").header("Authorization", "Bearer " + token))
                .andExpect(status().isOk())
```

```
                .andExpect(jsonPath("$.email").value(email))
                .andExpect(jsonPath("$.role").value("PASSENGER"));

        // PASSENGER không vào được khu vực admin
        mvc.perform(get("/api/admin/users").header("Authorization", "Bearer " + token))
                .andExpect(status().isForbidden());
    }

    @Test
    void wrongCodeIsLimited() throws Exception {
        String email = "binh.tran@example.com";
        mvc.perform(post("/api/auth/register").contentType(MediaType.APPLICATION_JSON)
                        .content("""
                                {"email":"%s","password":"MatKhau@123","fullName":"Trần Bình"}
                                """.formatted(email)))
                .andExpect(status().isAccepted());

        String wrong = "{\"email\":\"%s\",\"code\":\"000000\"}".formatted(email);
        for (int i = 0; i < 4; i++) {
            mvc.perform(post("/api/auth/verify-email").contentType(MediaType.APPLICATION_JSON).conte
                    nt(wrong))
                    .andExpect(status().isBadRequest());
        }
        // Lần sai thứ 5 (max-failed-attempts=5) → khoá mã
        mvc.perform(post("/api/auth/verify-email").contentType(MediaType.APPLICATION_JSON).content(w
                rong))
                .andExpect(status().isTooManyRequests());
    }

    private static String loginJson(String email) {
        return "{\"email\":\"%s\",\"password\":\"MatKhau@123\"}".formatted(email);
    }

    private static String extractCode(String body) {
        Matcher m = CODE.matcher(body);
        assertThat(m.find()).as("mail phải chứa mã 6 số").isTrue();
        return m.group(1);
    }
}
```

## 9.5 `security/RbacTest.java`: ma trận phân quyền

**`src/test/java/com/metroticket_api/security/RbacTest.java`**

```
package com.metroticket_api.security;

import static
        org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.j
        wt;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import java.util.List;
import java.util.UUID;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.context.annotation.Import;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.request.RequestPostProcessor;

import com.metroticket_api.TestcontainersConfiguration;

/** Kiểm tra ma trận phân quyền bằng JWT giả lập (không cần đăng nhập thật). */
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
@Import(TestcontainersConfiguration.class)
class RbacTest {

    @Autowired
    MockMvc mvc;
```

```
    private static RequestPostProcessor as(String role) {
        return jwt()
                .jwt(j -> j.subject(UUID.randomUUID().toString()).claim(TokenService.ROLES_CLAIM,
                        List.of(role)))
                .authorities(new RoleAuthoritiesConverter());
    }

    @Test
    void anonymousGets401() throws Exception {
        mvc.perform(get("/api/me")).andExpect(status().isUnauthorized());
        mvc.perform(get("/api/admin/users")).andExpect(status().isUnauthorized());
    }

    @Test
    void healthIsPublic() throws Exception {
        mvc.perform(get("/actuator/health")).andExpect(status().isOk());
    }

    @Test
    void onlyAdminCanManageUsers() throws Exception {
        mvc.perform(get("/api/admin/users").with(as("PASSENGER"))).andExpect(status().isForbidden())
                ;
        mvc.perform(get("/api/admin/users").with(as("STAFF"))).andExpect(status().isForbidden());
        mvc.perform(get("/api/admin/users").with(as("ADMIN"))).andExpect(status().isOk());
    }

    @Test
    void unknownRoleHasNoAccess() throws Exception {
        mvc.perform(get("/api/admin/users").with(as("HACKER"))).andExpect(status().isForbidden());
    }
}
```

### 9.6 Kết quả chạy thực tế

```
$ mvn -B test
Tests run: 4, Failures: 0, Errors: 0  -- in com.metroticket_api.security.RbacTest
Tests run: 2, Failures: 0, Errors: 0  -- in com.metroticket_api.auth.AuthFlowIntegrationTest
Tests run: 1, Failures: 0, Errors: 0  -- in com.metroticket_api.MetroticketApiApplicationTests
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Các tình huống đã được kiểm tra:

- Đăng ký → mail tới đúng người nhận, đúng tiêu đề, chứa mã 6 số.
- Chưa xác thực email thì đăng nhập bị `403`.
- Gửi lại mã trong 60 giây đầu bị `429` kèm `Retry-After`.
- Xác thực → đăng nhập → nhận JWT → `GET /api/me` trả đúng vai trò `PASSENGER`.
- Nhập sai 5 lần → `429` (bộ đếm được lưu dù request lỗi).
- Không có token → `401`. PASSENGER/STAFF vào `/api/admin/**` → `403`. ADMIN → `200`.
- Vai trò lạ trong token (`HACKER`) → không có quyền gì.
- `/actuator/health` công khai.

---

## 10. Thứ tự thực hiện và checklist

1. ☐ Tạo lại Maven wrapper (`mvn -N wrapper:wrapper`), sửa `.gitignore`.
2. ☐ Đổi tên 3 file sai quy ước (mục 1.3).
3. ☐ Thêm PostgreSQL vào `docker-compose.yml`, cập nhật `.env.example` và `.env`.
4. ☐ Thay `application.properties`, thêm `@ConfigurationPropertiesScan`.
5. ☐ Viết `V1`, `V2` migration.
6. ☐ Module mail: `AsyncConfig`, `TimeConfig`, `MailProps`, `MailMessage`, `MailSender`, `MailSendException`, `SmtpMailSender`, `MailTemplates`, template HTML.
7. ☐ Module verification: 10 file ở mục 5.
8. ☐ Phân quyền: package `user/` và `security/`.
9. ☐ API: package `auth/`, `ApiExceptionHandler`, `MeController`, `AdminUserController`.

10. □ Thêm dependency test, viết test, chạy `./mvnw -B verify`.
11. □ Sửa CI: thay placeholder `trivy-action@<commit-sha-da-kiem-tra>` bằng SHA thật; thay `<owner-viet-thuong>` trong `publish.yml`.
12. □ Gmail thật: bật xác minh 2 bước → tạo **App Password** → đặt vào `MAIL_PASSWORD`.

### Checklist bảo mật trước khi lên môi trường thật

- □ `JWT_SECRET`, `EMAIL_VERIFICATION_HASH_KEY` sinh ngẫu nhiên ≥ 32 ký tự, **khác nhau** giữa dev/staging/prod.
- □ `.env` không bị commit (CI đã có gitleaks).
- □ Đổi mật khẩu admin bootstrap, sau đó xoá `ADMIN_PASSWORD` khỏi môi trường.
- □ Log không chứa mã OTP, mật khẩu, token (đã che email, `toString()` của sự kiện không in mã).
- □ Chạy sau HTTPS (reverse proxy) để token không bị nghe lén.

---

## 11. Hạn chế và hướng mở rộng

| Hạn chế hiện tại | Hướng xử lý |
|---|---|
| Khoá tài khoản (`LOCKED`) không thu hồi token đã cấp; token vẫn dùng được tối đa 15 phút | Lưu `jti` bị thu hồi vào **Redis** (đã có trong compose) và kiểm tra trong một `OAuth2TokenValidator`; hoặc giữ TTL ngắn |
| Chưa có refresh token, người dùng phải đăng nhập lại sau 15 phút | Thêm bảng `refresh_token` (lưu hash, xoay vòng mỗi lần dùng) + endpoint `/ api/ auth/ refresh` |
| Giới hạn nhập sai tính theo mã, chưa theo IP | Rate limit theo IP cho `/ api/ auth/ **` (Bucket4j + Redis) |
| Mail gửi lỗi chỉ ghi log | Lưu bảng `outbox_mail` và retry định kỳ (Outbox pattern) |
| Vai trò, quyền cố định trong code | Nếu admin cần tự tạo vai trò, chuyển sang bảng `role`, `permission`, `role_permission` |
| `register` trả `409` khi email đã tồn tại, nên có thể dò được email | Luôn trả `202` và gửi mail thông báo "email đã được đăng ký" cho chủ email |